EASTERN INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTING AND INFORMATION TECHNOLOGY

EIU

TRƯỜNG ĐẠI HỌC
QUỐC TẾ
MIỀN ĐÔNG
EASTERN
INTERNATIONAL
UNIVERSITY

EDUTRACK: A SMART LEARNING CENTER MANAGEMENT SYSTEM

Huynh Khanh Duy – 2331200014

Au Duong Tai – 2331200205

Hoang Quoc Viet – 2331200183

Software Engineering

Ho Chi Minh City, August, 2026

EASTERN INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTING AND INFORMATION TECHNOLOGY

TRƯỜNG ĐẠI HỌC
QUỐC TẾ
MIỀN ĐÔNG
EASTERN
INTERNATIONAL
UNIVERSITY

EDUTRACK: A SMART LEARNING CENTER MANAGEMENT SYSTEM

Huynh Khanh Duy – 2331200014

Au Duong Tai – 2331200205

Hoang Quoc Viet - 2331200183

Software Engineering

Ho Chi Minh City, August, 2026

ABSTRACT

In the current educational landscape, tutoring centers and private educational institutions often struggle with inefficient administrative workflows. Relying on fragmented tools or manual processes for managing student data, tracking academic progress, and handling tuition payments leads to administrative bottlenecks and a lack of transparency between the center, students, and parents.

To address these challenges, this project introduces EduTrack, a Smart Learning Center Management System designed to centralize and automate daily educational operations.

As part of the Software Engineering Capstone Project I, the current phase focuses on comprehensive system analysis, architectural design, and UI/UX prototyping. EduTrack is structured to serve three primary user groups encompassing five distinct roles: Administration (Center Manager, Staff), Academic (Teacher, Teaching Assistant), and End-Users (Student). Rather than exhaustively detailing all system features, the core modules designed in this phase highlight critical operations, including centralized user and student management, class and subject management, tuition and approval workflows, teaching and student progress monitoring, notification management, and reporting. Furthermore, the system architecture incorporates AI-powered integrations designed to provide smart educational support, thereby enhancing both teaching efficiency and student learning experience.

By establishing a solid architectural foundation and clear logic workflows, the proposed system aims to deliver comprehensive benefits across all user levels. For administrators, it significantly reduces manual workloads and streamlines center operations. For teaching staff, the integration of AI and automated tools simplifies class management and evaluation processes. For end-users, it guarantees transparent, real-time insights into learning outcomes and financial records. Ultimately, the deliverables of this phase serve as a robust blueprint for the full technical implementation in Capstone Project II, III.

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our lecturer, Mr. Ung Van Giau, for providing valuable knowledge, guidance, and support throughout the Software Engineering Capstone Project I.

We are deeply grateful to the lecturers who share insights regarding academic administration and student progress tracking workflows, which greatly contributed to the design and development of the EduTrack system, with us.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

No.	Term	Meaning
1	API	Application Programming Interface
2	ERD	Entity-Relationship Diagram
3	JWT	JSON Web Token
4	TA	Teaching Assistant
5	UI	User Interface
6	UX	User Experience
7	LLM	Large Language Model
8	CRM	Customer Relationship Management

CHAPTER 1. INTRODUCTION

This chapter presents the primary motivation driving the need for a centralized learning center management system, outlines the core objectives, and defines the scope of the project. Finally, it concludes with a brief outline of the organization of the remaining chapters in this report.

1.1. Motivation

Currently, many individual teachers or groups of teachers have opened classes based on a tutoring center model to organize teaching activities in a more systematic and professional manner [1]. However, this transition introduces new challenges, such as managing students, tracking learning progress, scheduling classes, and controlling tuition payments. As a result, managing records via Excel, messaging apps, or personal notes can be time-consuming and lead to tracking errors, including confusion over student information, class schedules, classes, teachers, and teaching assistants. Therefore, these centers require a centralized system to manage students, classes, subjects, teachers, and tuition fees clearly and transparently [2]. This is a practical need for growing centers that still depend on manual methods. The EduTrack project is developed to support remote, centralized information management, aiming to reduce manual workloads and minimize errors.

1.2. Project Objectives

The primary objective of this project is to design and develop **EduTrack**—a comprehensive, centralized Web Application for learning center management tailored specifically to the operational needs and financial constraints of growing, small-to-medium tutoring centers. The project is structured around four primary objective pillars, aligning the system's technical design with practical operational requirements:

- Cross-browser and Platform Compatibility: Ensure that the web-based system operates seamlessly across all major modern web browsers, including Google Chrome, Safari, Microsoft Edge, and Firefox. This compatibility guarantees zero service interruption and consistent functionality regardless of the client's local operating system or browser software.
- Responsive Design: Deliver a highly responsive, modern, and accessible user interface using React and TypeScript. The frontend layouts must dynamically adapt to various screen sizes and device viewports—including desktop computers, tablets, and mobile phones—to optimize the usability and user experience (UX) for all five system roles.
- Core Functional Delivery: Provide a robust digital platform for core academic and administrative operations. This includes user management, class management, class operations, reporting, approval request, and role-based access control. AI Assistance Integration is also

prioritized as a core educational helper. The system is designed to assist teachers by automatically generating suggested scores and qualitative feedback. While the specific LLM API provider and connection SDKs will be finalized in Capstone II and III, the database design and backend routing are fully established in this phase. Moreover, to prevent scope creep and ensure technical design depth, the Tuition module in this phase is strictly focused on Billing and Invoice Generation (Billing only), while the Notification module is strictly limited to static, in-app database-driven alerts. Complex third-party integrations—specifically automated notification gateways (such as SMS, Zalo ZNS, or SMTP email APIs) and online payment gateways (such as VNPAY or MoMo)—are explicitly designated as Supporting Features to be explored and developed in future phases as time permits.

– Deliver complete system architecture, database schemas, and UI prototypes as the foundation for Capstone Project II and III.

1.3. Project Scope

The scope of this project is strictly defined according to the academic requirements of Capstone Project I. Rather than delivering a fully coded and deployed application, the boundary of this current phase focuses exclusively on foundational software engineering processes, including system analysis, architectural design, and prototyping. Specifically, the scope of work encompasses system requirement analysis to elicit and define functional and non-functional requirements across all designated user roles. It also involves designing the relational database architecture, utilizing Entity-Relationship (ER) diagrams to securely manage entities such as users, classes, assignments, and tuitions. Furthermore, this phase includes mapping out detailed system workflows for core business processes, alongside analyzing the essential pages and core features required for the system**.** To visualize the system layout, the scope also covers UI/UX prototyping through the design of key interface mockups. Consequently, full-scale technical coding, system deployment, and complex third-party API integrations—such as AI models or live payment gateways—fall outside the scope of Capstone Project I and are reserved for future implementation in Capstone Project II, III.

1.4. Organization of the project

- Chapter 1 — Introduces motivation, objectives, scope, and organization of the project.
- Chapter 2 — Provides information about related works.
- Chapter 3 — Describes the requirements and analyzes the system design.
- Chapter 4 — Provides static UI mockups.
- Chapter 5 — Shows conclusion, and future works

CHAPTER 2. RELATED WORKS

This chapter shows a review of existing educational management systems to identify their limitations and highlight the practical need for EduTrack.

2.1. TutorBird

TutorBird [3] is a widely recognized management platform tailored for private tutors and tutoring centers. It focuses on simplifying scheduling, billing, and student management.

– Strengths**:** The platform offers a highly user-friendly interface accompanied by dynamic page transitions and comprehensive documentation, including video tutorials. It streamlines administrative tasks by supporting data importing (via Excel/CSV), calendar-based scheduling, and online payments. Furthermore, it provides automated reminder notifications for fixed payment due dates and offers sample assignment templates for teachers.

– Limitations**:** Despite its usability, TutorBird presents several rigid constraints. Creating a student profile strictly requires parent information, which may not always be applicable. Financial operations are also inflexible; tuition fees are fixed upon setup and difficult to adjust mid-course, and the payment gateway is restricted to a single currency (USD). Additionally, users frequently experience slow data loading times during page transitions.

2.2. Easy Edu

Easy Edu [4] is a comprehensive educational management software popular in the Vietnamese market, designed to support various educational models including online, offline, and hybrid tutoring.

– Strengths**:** The system significantly optimizes management workflows and human resource operations, claiming to reduce management time by 80% and administrative costs by 50%. It features a built-in ecosystem for online teaching and testing, and an advanced CRM (Customer Relationship Management) for customer care, and automates up to 90% of financial and accounting tasks. Furthermore, it ensures robust data security through three-layer authorization and firewall protection.

– Limitations**:** The primary drawback of Easy Edu is its pricing model. The service requires a daily maintenance fee calculated per user (e.g., 2,000 VND/day/user). Consequently, operational costs increase linearly and significantly as the center expands its staff and student base, creating a financial burden for newly emerging centers.

2.3. EduSpace

EduSpace [5] is another robust centralized management platform that emphasizes communication, flexible billing, and E-learning integration.

– Strengths**:** EduSpace boasts an intuitive and simple interface that strictly manages student statuses, including attendance, course reservations, and makeup classes. It offers highly flexible tuition processing with automated payment reminders integrated via Zalo and Email. The platform also supports diverse salary calculation mechanisms, E-learning capabilities (course creation, online exams), and allows for unlimited staff members and facility branches.

– Limitations**:** Similar to Easy Edu, the limitations of the EduSpace lie in its commercial subscription model. The platform is divided into various tiered service packages. Costs escalate rapidly when the number of students or CRM contacts exceeds the package limits. Furthermore, it mandates annual billing, which imposes a high initial financial burden on smaller tutoring centers.

2.4. Moodle

Moodle [6] is a prominent open-source Learning Management System designed for delivering educational content, facilitating interactive online courses, and rigorously evaluating student academic progress in a centralized digital environment.

– Strengths: As a free, open-source platform, Moodle offers extensive customization without costly software licenses. A large global community provides thousands of free add-ons, easily enabling advanced functions like detailed quizzes and video meeting integration. Additionally, it provides excellent support for teaching through reliable grading systems and clear student progress tracking.

– Limitations: The main drawback of Moodle is its high demand for technical skills, requiring a dedicated IT team for server hosting, initial setup, and regular maintenance. Moreover, the platform is not designed to function as a complete business management system for educational centers. Because it lacks essential administrative tools for scheduling classes, calculating staff pay, and processing tuition fees, it is highly impractical for daily operations.

2.5. Summary and The Need for EduTrack

Existing systems like TutorBird, Easy Edu, and EduSpace provide solutions for managing educational and tutoring centers with a variety of features such as student management, scheduling, attendance, billing, and online payment for TutorBird, as well as broader functions for student management, tuition, classes, financial operations, and other administrative activities for Easy Edu and EduSpace. Meanwhile, open-source LMS platforms like Moodle excel in content delivery and academic tracking but lack built-in center management and business operational tools. Furthermore, commercial systems differ significantly in pricing models, operational scope, and target users; Easy Edu adopts a user-based pricing model, while EduSpace provides tiered plans based on the number of students. These differences show that

existing solutions may not be able to meet the specific operational demands and management model targeted by this project.

Therefore, **EduTrack** is proposed as a tailored, cost-effective, and centralized solution. It focuses strictly on the core administrative necessities-student tracking, assignment management, attendance, and basic tuition control without the financial overhead or feature bloat of commercial alternatives.

CHAPTER 3. SYSTEM ANALYSIS & DESIGN

This chapter details the system requirements of the EduTrack system. It also presents use case, system architecture, and activity diagrams to visualize user interactions and system workflows. Furthermore, it provides a complete database design, including Entity-Relationship Diagrams (ERD) and detailed table descriptions. Finally, it outlines the essential pages and core features required for the system to function effectively.

3.1. System Requirements

This section defines the core capabilities that the EduTrack system must possess to meet the needs of all users. It is divided into Functional Requirements, which describe specific system behaviors for each user role, and Non-functional Requirements, which define the performance and quality standards of the system.

3.1.1. Functional Requirements

To ensure clear organization, the functional requirements are grouped based on the five primary user roles: Center Manager, Admin Staff, Teacher, Teaching Assistant (TA), and Student/Parent.

Group 1: Center Manager, this role has the highest level of access. Table 1 outlines the requirements for managing the entire center, including accounts, academic structures, and financial policies.

Table 1. Center Manager Requirements

ID	Feature	Description
CM-01	Account Management	Allows the Center Manager to create, update, lock, or unlock user accounts, and assign or revoke their roles.
CM-02	Academic Management	Enable the Center Manager to create and manage subjects and classes, and assign teachers, TAs, and students to them.
CM-03	Center Dashboard	Provides a dashboard to monitor overall statistics regarding students, classes, staff, and tuition.
CM-04	Student Monitoring	Allow the Center Manager to monitor student attendance, assignments, and learning progress.
CM-05	Reporting	Enables the Center Manager to view and export detailed reports on students, classes, tuitions, and revenue.
CM-06	Tuition Configuration	Allows the Center Manager to set tuition fees, collection dates, deadlines, and discount policies for each subject or class.
CM-07	Global Announcements	Allows the Center Manager to send notifications to the entire center, specific classes, or target user groups.
CM-08	Center Settings	Enable the Center Manager to update the center's general information and system settings.

CM-09	Request Approval	Allows the Center Manager to approve or reject special adjustment requests (e.g., fee waivers, refunds, special class transfers) submitted by Admin Staff.

Group 2: Admin Staff handle daily operations, customer support, and basic financial tracking.

Table 2. Admin Staff Requirements

ID	Feature	Description
AD-01	Profile Management	Allows Admin Staff to create and update student and parent profiles, and link parents to their children.
AD-02	Search Profiles	Enables Admin Staff to search and view student profiles by name, ID, class, or learning status.
AD-03	Enrollment Operations	Allows Admin Staff to enroll students into classes, transfer them, or update their status (e.g., paused, finished).
AD-04	Class Information	Allows Admin Staff to view class schedules, assigned teachers, and update class details within their permission scope.
AD-05	Tuition Recording	Allows Admin Staff to record tuition payments, including the amount, payment date, method, and notes.
AD-06	Tuition Status	Enables Admin Staff to update and track tuition statuses (unpaid, paid, or overdue).
AD-07	Notifications	Allows Admin Staff to send notifications regarding schedules, class changes, and tuition reminders to students and parents.
AD-08	Basic Reporting	Allows Admin Staff to view and export basic reports regarding students and tuition collection.
AD-09	Feedback Handling	Allows Admin Staff to receive, record, and respond to feedback or requests from students and parents.
AD-10	Escalate Requests	Enables Admin Staff to submit special requests (fee waivers, refunds) to the Center Manager for approval.
AD-11	Teaching-Monitoring	Allows Admin Staff to monitor teaching activities by teachers, class subject, attendance, assignment completion, and student learning progress.

Group 3: Teachers focus on class management, grading, and student progress tracking.

Table 3. Teacher Requirements

ID	Feature	Description
TC-01	Account Login	Allows Teachers to securely log into the system and recover forgotten passwords via email.
TC-02	View Classes	Allows Teachers to view the details and schedules of their assigned classes.
TC-03	Attendance	Enables Teachers to take and manage student attendance for their classes.

TC-04	Assignment Management	Allows Teachers to create, assign, and grade homework for students.
TC-05	Progress Tracking	Allows Teachers to track student learning progress and send academic feedback to parents.
TC-06	Issue Reporting	Enables Teachers to report schedule issues or class problems to the Admin Staff.
TC-07	AI Assistance	Integrates LLM tools to support Teachers in automatically grading standard assignments.

Group 4: Teaching Assistants (TA) support the teachers in managing the classroom and checking assignments.

Table 4. Teaching Assistant Requirements

ID	Feature	Description
TA-01	Account Login	Allows TAs to securely log into the system and recover forgotten passwords via email.
TA-02	View Classes	Allows TAs to view the details and schedules of their assigned classes.
TA-03	Assist Attendance	Allows TAs to assist in taking student attendance.
TA-04	Class Notes	Enables TAs to write and save notes regarding the daily class situation.
TA-05	Assist Grading	Allows TAs to check and grade specific assignments if granted permission by the Teacher.
TA-06	Issue Reporting	Allows TAs to report class issues directly to the Teacher or Admin Staff.

Group 5: This group represents the end-users who consume the center's services.

Table 5. Student and Parent Requirements

ID	Feature	Description
ST-01	Account Access	Allows Students/Parents to log in with admin-provided accounts and recover passwords via phone or the parent app.
ST-02	View Information	Allows Students/Parents to view their class schedules, and view or update basic personal information.
ST-03	Submit Assignments	Enables Students to view assigned homework and submit their answers online.
ST-04	View Progress	Allows Students/Parents to view academic progress and read feedback from Teachers.
ST-05	Receive Notifications	Enables Students/Parents to receive system notifications from classes or the center.

ST-06	Tuition Payment	Enables Students and Parents to view their current tuition status and make online tuition payments.

3.1.2. Non-functional Requirements

Beyond the specific features, the system must maintain a high standard of quality, security, and usability. Table 6 outlines the non-functional requirements that guide the system's technical architecture.

Table 6. Non-functional Requirements

ID	Category	Description
NFR-01	Security	The system shall securely hash user passwords and use JWT for authenticated sessions.
NFR-02	Usability	The user interface of the system shall be responsive and function seamlessly across desktop computers, tablets, and mobile phones.
NFR-03	Compatibility	The system shall be compatible with all major modern web browsers (Google Chrome, Safari, Microsoft Edge, Firefox).

3.2. Use Case Diagram

3.2.1. Actors

Main actors include:

- Center Manager
- Admin Staff
- Teacher
- Teaching Assistant
- Student

3.2.2. Use Cases

General use cases include:

- Manage Center Operations
- Manage Accounts
- Process Finances
- Execute Learning Activities
- Handle Communications
- Handle Administrative Tasks
- Manage User Profile
- Configure System

3.2.3. General Use Case Diagram

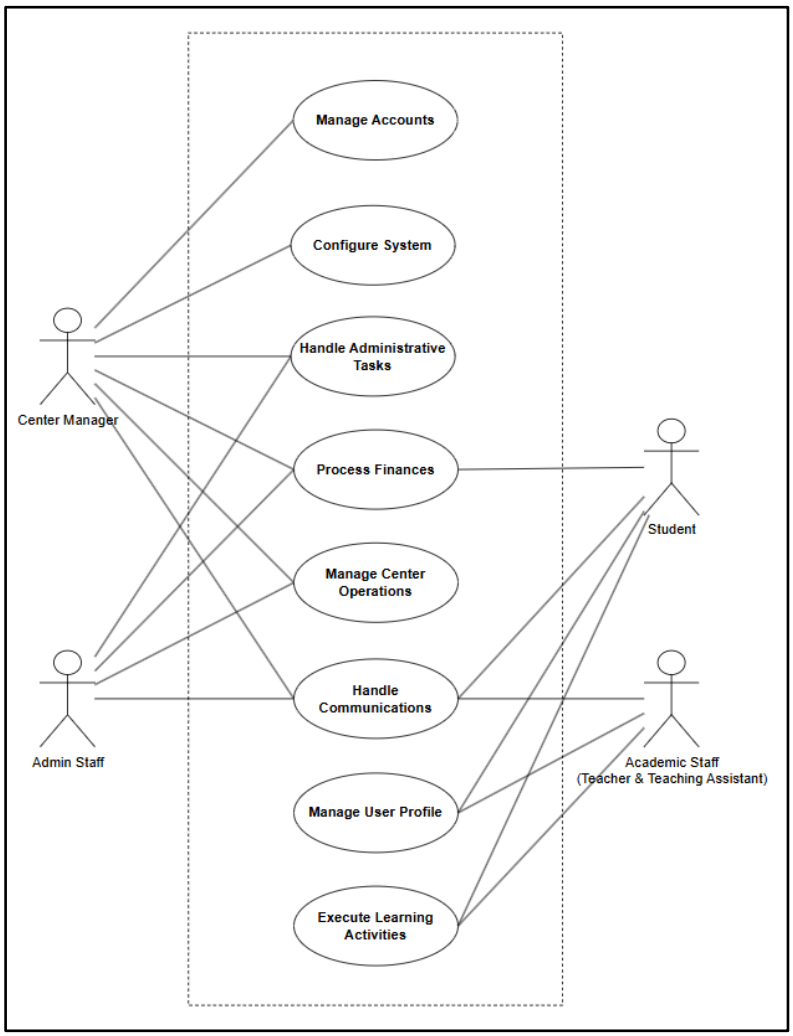

Figure 1. General Use Case Diagram

Center Manager: As the top administrator, the Center Manager has full control over the system and can also perform all tasks assigned to the Admin Staff.

- Configure System: Sets up general system rules and manages settings for the entire center.
- Manage System Accounts: Creates, updates, and manages accounts and access rights for everyone in the system.
- Process Workflows: Reviews and approves internal staff requests. This actor also views general performance reports.
- Handle Communications: Sends, receives, and manages messages for the whole center or specific groups.

Admin Staff: The Admin Staff handles the daily office work and financial tasks of the center.

- Manage Center Operations: Manages student records, subjects, class schedules, and checks on daily teaching activities.
- Process Finances: Handles money matters by creating bills, sending reminders, and keeping track of tuition records.
- Process Workflows: Sends internal requests for the Center Manager to approve and views system reports.
- Handle Communications: Manages daily messages and announcements for staff and clients.

Academic Staff (Teacher & Teaching Assistant): Teacher and Teaching Assistant focus on teaching, grading, and keeping track of student progress.

- Execute Learning Activities: Use the system to view assigned classes, teaching schedule, create and assign homework, grade student work, and check overall students performance.
- Manage Personal Profile: View and update their own personal information and account settings.
- Handle Communications: Send class-related messages to students and receive updates from the administration.

Student: The Student uses the system to follow classwork and pay tuition fees.

- Execute Learning Activities: Checks learning resources, submits homework, and views learning progress.
- Process Finances: Views bills and pays tuition fees through the system.
- Manage Personal Profile: Views and updates their own personal details and account settings.
- Handle Communications: Receives messages about schoolwork, payment due dates, and general center news.

3.3. Overall System Architecture

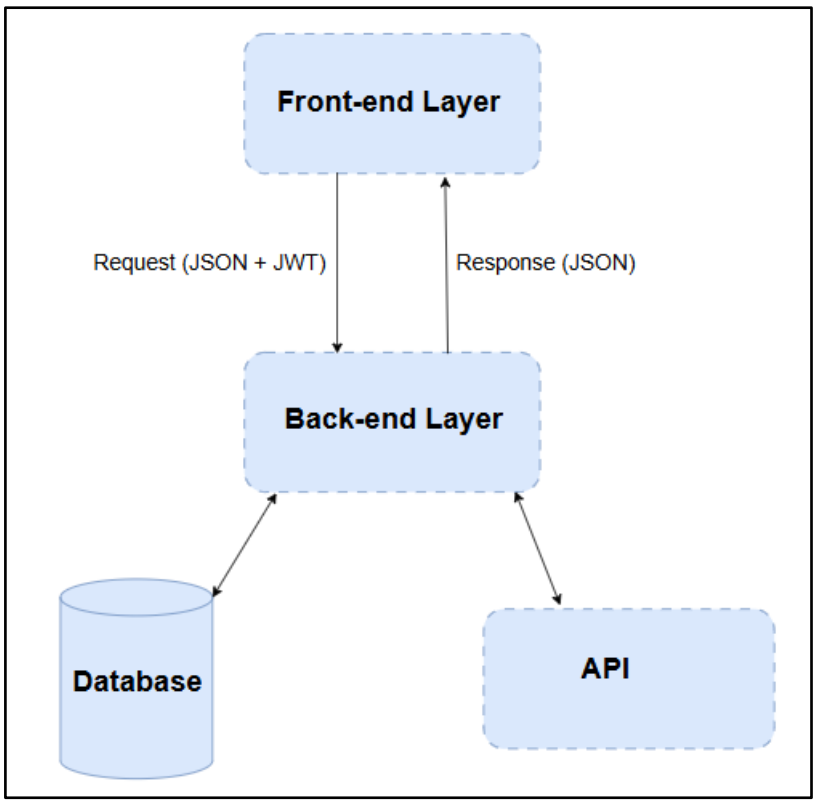

Figure 2. Overall System Architecture

3.3.1. Architectural Overview

The proposed system is a web-based Learning Center Management System designed to support the management and operation of a learning center. The system provides functions for managing staff, students, courses, assignments, submissions, grades, and learning-related information. In addition, a Large Language Model (LLM) API is integrated to provide AI-assisted grading and feedback.

The system adopts a layered three-tier architecture consisting of the presentation layer, application/backend layer, and data layer. An additional LLM integration component is implemented within the backend to communicate with external Large Language Model services.

The architecture was selected to improve maintainability, scalability, security, and separation of concerns. Each layer is responsible for a specific set of functions and communicates with other layers through well-defined interfaces.

3.3.2. Presentation Layer

The presentation layer is implemented using React and TypeScript. It provides the user interface through which administrators, teachers, and students interact with the system.

The main functions of the presentation layer include:

- User authentication and session management

- Student, staff, and teacher profile management
- Class scheduling, session, and attendance tracking
- Assignment, submission, and grading workflows
- AI-assisted grading and feedback review
- Dashboard statistics and analytical reporting

React components are organized into reusable components, pages, layouts, and feature-specific modules. TypeScript is used to provide static type checking and improve code reliability and maintainability.

The Front-end communicates with the backend through RESTful APIs over HTTPS. JWT Bearer Tokens are attached to authenticated API requests to ensure that users can only access functions permitted by their roles.

3.3.3. Application and Backend Layer

The backend is developed using ASP.NET Core Web API. It provides RESTful APIs for business operations and acts as the main communication layer between the React application, MySQL database, and external LLM services.

The backend is organized into several logical components:

- Controllers: Controllers receive HTTP requests from the frontend, validate request parameters, and invoke appropriate business services.
- Service Layer: The service layer contains the main business logic of the system. Examples include StudentService, StaffService, CourseService, AssignmentService, AttendanceService, GradingService, and UserService.
- Data Access Layer: The data access layer manages communication with the MySQL database using Entity Framework Core. It is responsible for retrieving, creating, updating, and deleting persistent data.
- API Security & Authorization: Coordinates with the security middleware to intercept incoming requests, validate JWT signatures, and enforce role-based access before business logic execution.

This separation of responsibilities allows the backend to be easier to test, maintain, and extend.

3.3.4. Data Layer

MySQL is used as the primary relational database management system. The database stores the persistent data required by the learning center management system. The main entities may include:

- User, Role, and UserRole (Account & Authorization)

- Subject, Class, TeacherClass, TaClass, and StudentClass (Academic & Enrollments)
- ClassSchedule and ClassSession (Scheduling & Timetables)
- ContentUnit, Attendance, and ProgressNote (Learning & Tracking)
- Assignment and Submission (Coursework & Evaluations)
- TuitionInvoice and Payment (Financial Operations)
- ApprovalRequest (Administrative Approvals)
- Notification and CenterSetting (Communication & Configurations)

Entity Framework Core is used as the Object-Relational Mapping (ORM) framework between ASP.NET Core and MySQL. This approach allows application entities to be represented as strongly typed C# classes while reducing the amount of manually written SQL code.

3.3.5. Authentication and Authorization

Security is an important aspect of the proposed system because the application manages personal information, academic records, assignments, and grades.

The system uses JSON Web Token (JWT) based authentication. After successful login, the Back-end generates a JWT containing relevant user information and authorization claims. The Front-end uses the token when accessing protected API endpoints.

The authentication process can be summarized as follows:

1. The user submits login credentials through the React application.
2. The ASP.NET Core API validates the credentials against the hashed records.
3. If the credentials are valid, the server generates a JWT.
4. The frontend stores and uses the token for authenticated API requests.
5. The backend validates the JWT signature and claims for each protected request.

Role-based authorization determines whether the user is allowed to perform the requested operation. For example, an administrator may manage staff and students, while a teacher may create assignments and grade submissions. Students and parents may access their courses, assignments, submissions, and grades but cannot modify grades.

Passwords are never stored as plain text; a secure password hashing mechanism is used for credential storage.

3.3.6. LLM Integration

The system integrates a Large Language Model API to provide AI-assisted grading and feedback.

The LLM integration is implemented as a separate backend service rather than allowing the React application to communicate directly with the LLM provider. This design prevents

API credentials from being exposed to client-side code and allows the backend to control prompts, validation, logging, and access policies.

For an AI-assisted grading request, the backend provides the LLM with:

- Assignment description
- Grading rubric
- Maximum score
- The answer or submission of student
- Assessment criteria and additional grading instructions

The LLM returns a structured grading result containing a suggested score and feedback. The backend validates and stores the result before presenting it to the teacher.

The proposed workflow follows a human-in-the-loop approach. The AI provides a suggested grade and feedback, while the teacher reviews and confirms the final result. This approach reduces the risk of relying entirely on automatically generated grades.

3.4. Technologies

Table 7. Technology Stack

Category	Technology	Description
Front-end	HTML5, CSS3, React, TypeScript	Component-based library and strongly typed language used to build scalable, interactive user interfaces.
Back-end	ASP.NET Core 8.0 (C#)	Cross-platform framework used to implement backend services, routing, and business logic.
Database	MySQL	Relational database management system used for persistent data storage.
Authentication	JWT (JSON Web Token)	Stateless authentication mechanism used to secure API endpoints and user sessions.

3.4.1. React TypeScript

For the Front-end, we specifically chose React [7] with TypeScript [8] (React TS) to create a modern and user-friendly interface. While React allows us to build reusable UI components efficiently, using plain JavaScript can sometimes lead to unexpected errors. TypeScript solves this by providing static typing. This helps us catch code errors early during development, making the source code much cleaner, safer, and easier to maintain in the long run.

3.4.2. ASP.NET Core Web API

The Back-end of the EduTrack system is built using ASP.NET Core Web API [9]. We chose this framework because it provides a strong and secure way to handle all the business

logic, such as managing students, teachers, classes, and tuition fees. It is fast and works perfectly with Entity Framework, which makes saving and updating database records much easier. By separating the backend from the Front-end, we can develop, test, and maintain the system's logic without affecting the user interface.

3.4.3. Database Design

To store all the system data, we use MySQL [10]. While SQL Server is a common choice for .NET projects, it can cause difficulties during real-world deployment due to high licensing costs. Therefore, we chose MySQL because it is a reliable, open-source, and free relational database that is perfect for managing connected data in a learning center. To connect our ASP.NET Core backend with the MySQL database, we use Pomelo, which is a highly optimized and widely used provider for this setup. During the development phase, we run MySQL Community Server locally and use MySQL Workbench to easily design database tables.

3.4.4. Authentication

For user authentication, we use JSON Web Tokens (JWT) [11]. We chose JWT because it is lightweight, stateless, and integrates easily between React and ASP.NET Core. Instead of storing user sessions on the server, the client sends a secure token with each request to verify the user's identity and role (such as student, teacher, or admin). This keeps backend server memory low, improves response speed, and ensures safe access to sensitive endpoints across the EduTrack system.

3.5. Database Design

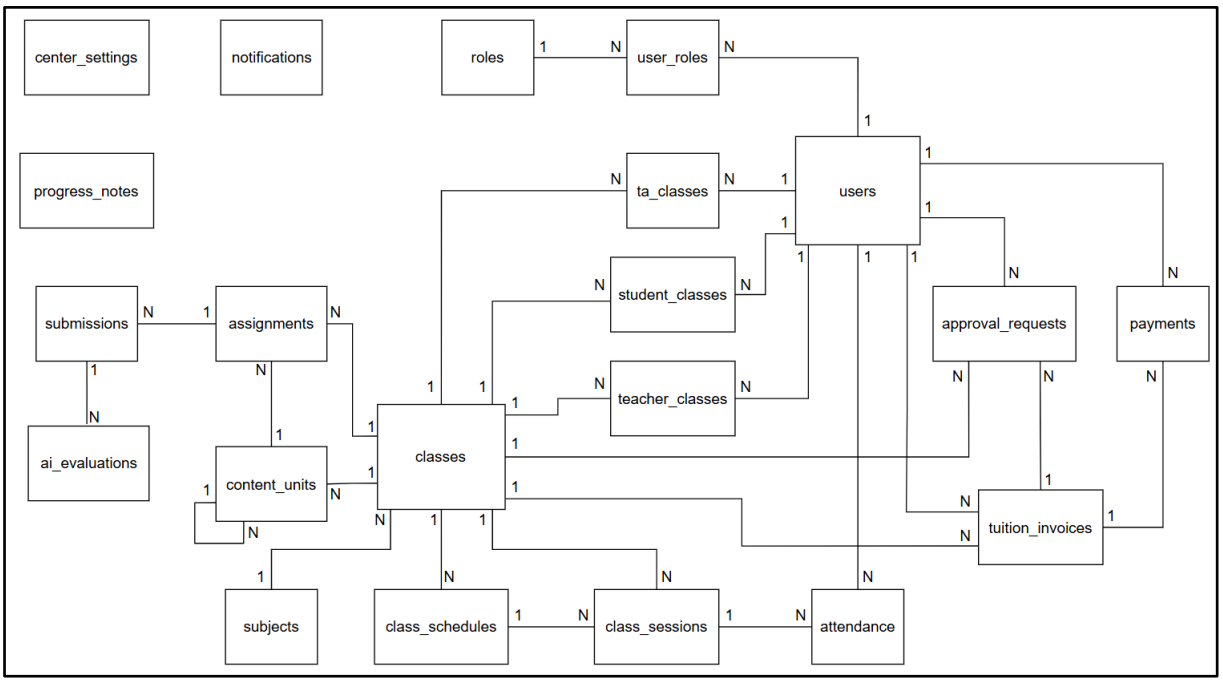

Figure 3. General ERD

This system-wide ERD connects users and their assigned roles to classes and subjects, integrates curriculum and evaluation via content_units, assignments, and submissions, manages operational workflows like class_schedules, class_sessions, and attendance, and supports administrative processes through tuition_invoices, payments, and approval_requests.

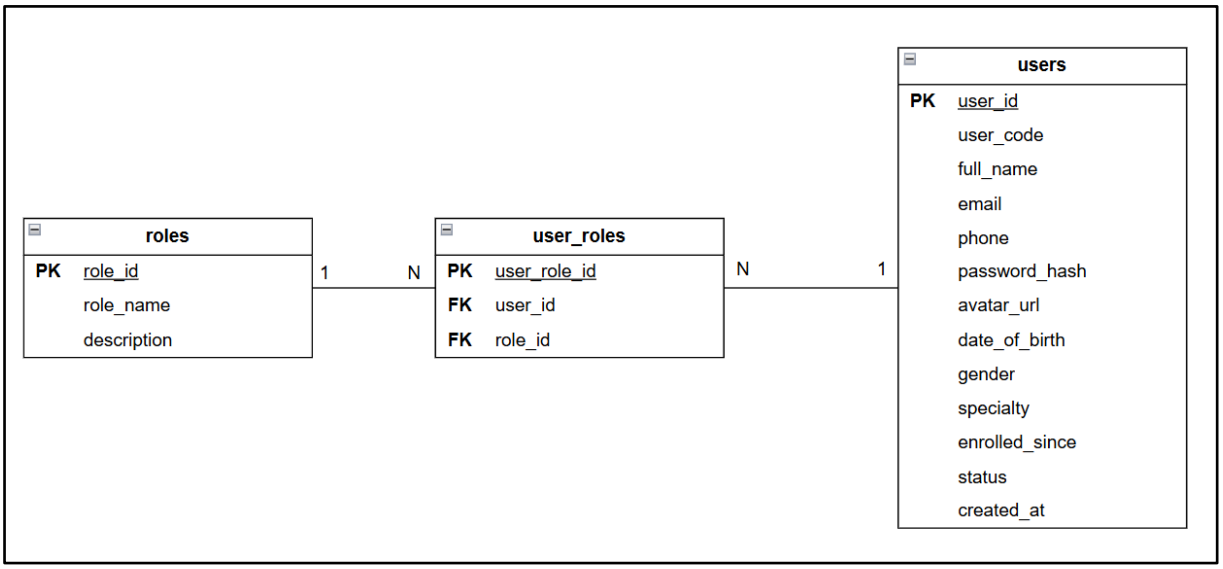

Figure 4. User Authentication and Role Management ERD

The EduTrack user management structure uses Role-Based Access Control (RBAC). The users table stores common account information, while roles define system roles and user_roles manage the many-to-many relationship between users and roles.

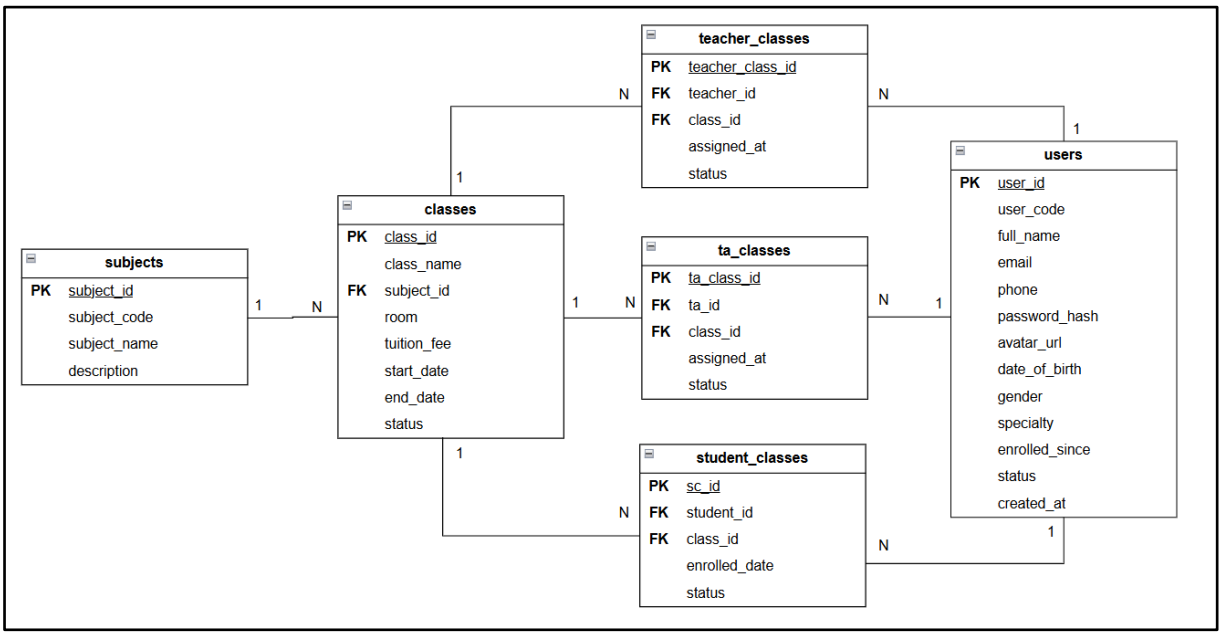

Figure 5. Academic Structure and Class Enrollment ERD

The academic structure connects subjects with classes and manages class participation through dedicated assignment tables. Teachers, teaching assistants, and students can be associated with multiple classes through teacher_classes, ta_classes, and student_classes.

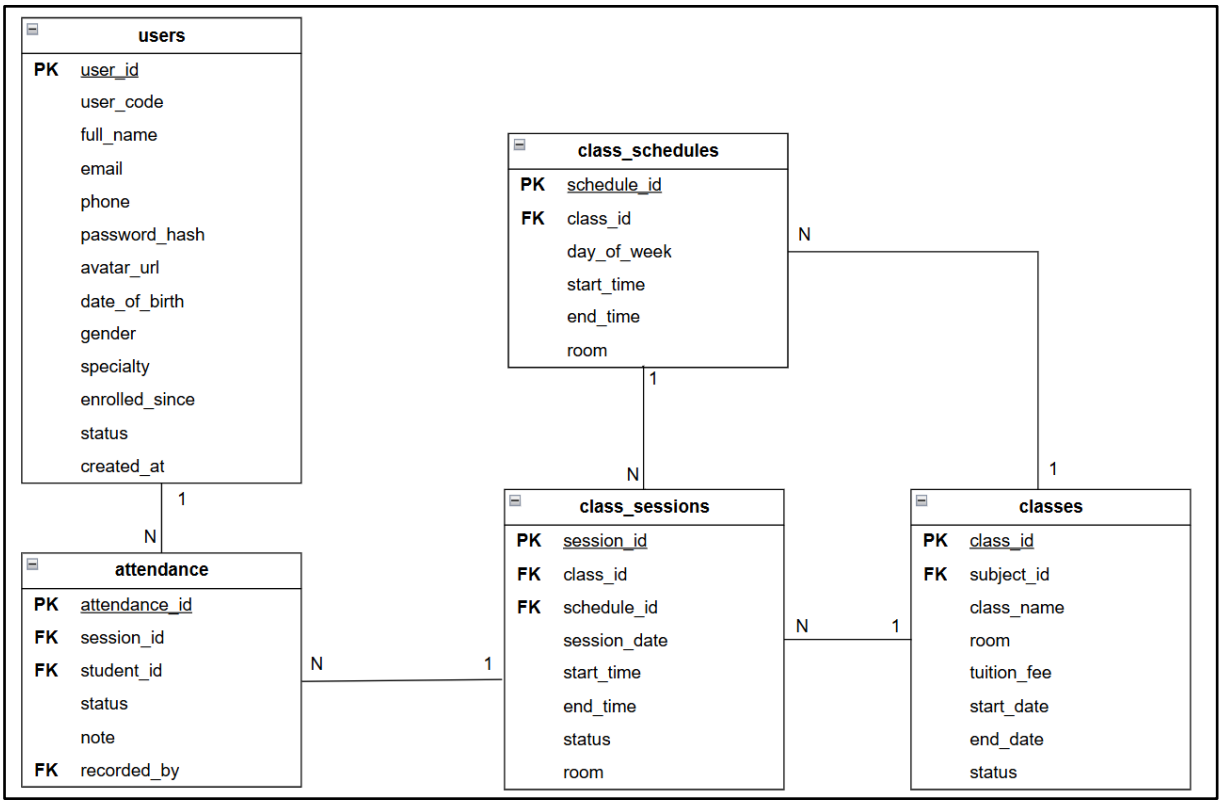

Figure 6. Class Scheduling, Session, and Attendance ERD

The scheduling structure separates recurring class schedules from actual teaching sessions. Each class may have multiple scheduled time slots and individual sessions, while attendance is recorded for each student based on a specific session.

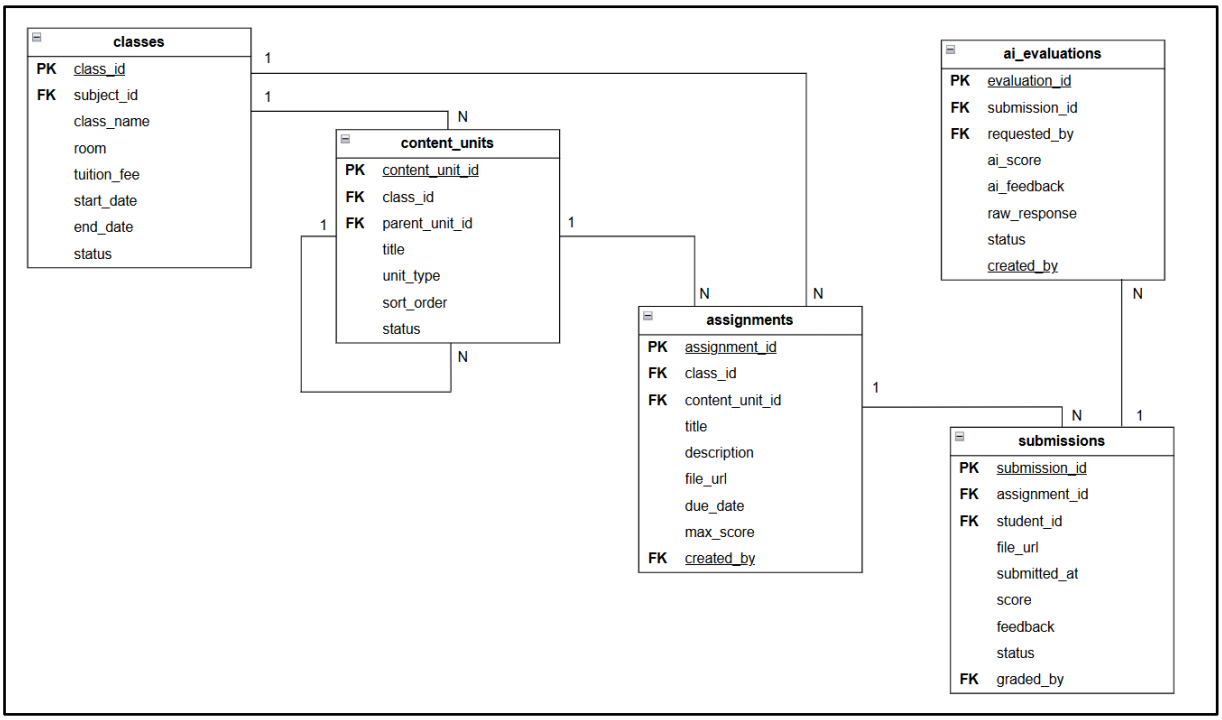

Figure 7. Learning Content and Assignment Management ERD

Learning content is organized through the self-referencing content_units table, allowing hierarchical structures such as chapters and lessons. Assignments are associated with classes and may optionally be linked to a specific content unit, while student submissions are recorded separately.

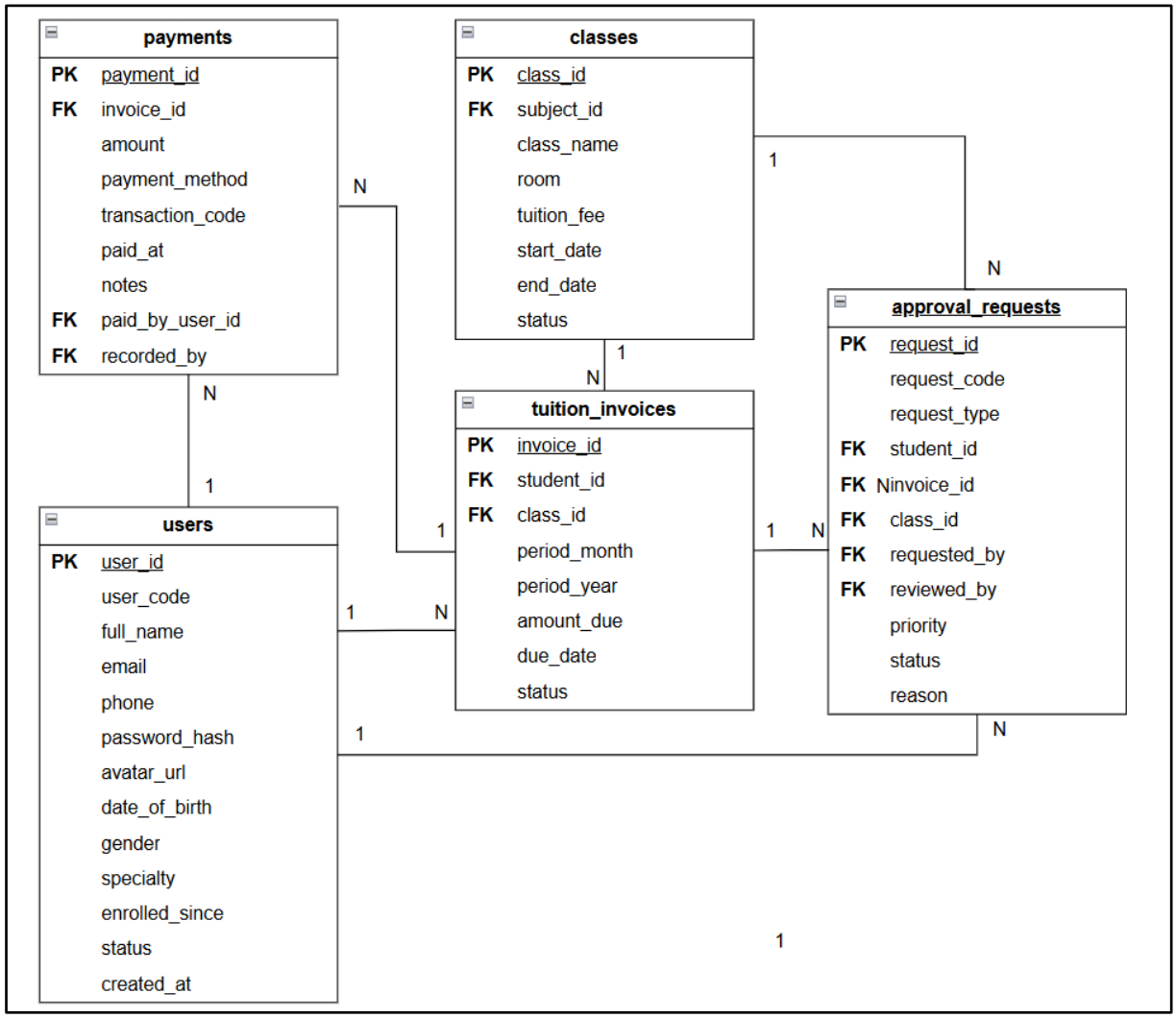

Figure 8. Tuition Payment and Approval Management ERD

The tuition structure manages student invoices, payment records, and exceptional approval requests. Each invoice may contain multiple payment records, while approval requests support tuition-related and class-related exceptions that require managerial review.

Table 8. Users table

Attribute	Data Type	Key	Constraints	Description
user_id	INT	PK	AUTO_INCREMENT, NOT NULL	Unique internal identifier for the user record
user_code	VARCHAR(20)		UNIQUE, NOT NULL	Unique account login code / username assigned by Admin (e.g., MGR001, TC001, STU001)
full_name	VARCHAR(150)		NOT NULL	Full legal name of the user

email	VARCHAR(150)		UNIQUE, NULL	User's email address (can also be used as a login identifier)
phone	VARCHAR(20)		NULL	Contact telephone number (can also be used for identification)
password_hash	VARCHAR(255)		NOT NULL	Cryptographically hashed password provisioned by Admin
avatar_url	VARCHAR(500)		NULL	URL pointing to the user's profile image
date_of_birth	DATE		NULL	Date of birth
gender	ENUM		'male', 'female', 'other'	Gender of the user
specialty	VARCHAR(100)		NULL	Academic subject specialty for teachers (e.g., Mathematics, Physics)
enrolled_since	DATE		NULL	Enrollment date for students or hiring date for staff/teachers
status	ENUM		'active', 'inactive', 'on_hold', 'locked', Default: 'active'	Current operational status of the account
created_at	DATETIME		NOT NULL, Default: CURRENT_TIMESTAMP	Timestamp when the account was provisioned by Admin

Description: Centralized entity storing user account credentials, authentication details, and personal profiles for all system roles (Center Managers, Admin Staff, Teachers, Teaching Assistants, and Students). All accounts and initial passwords are created and provisioned directly by the Center Manager.

Table 9. Roles table

Attribute	Data Type	Key	Constraints	Description
role_id	INT	PK	AUTO_INCREMENT, NOT NULL	Unique internal identifier for the role
role_name	VARCHAR(50)		UNIQUE, NOT NULL	Name of the role
description	TEXT		NULL	Detailed description of the role's responsibilities and permission scope

Description: Master lookup table storing system authority roles. It defines the operational scopes for access control and can be dynamically extended by administrators.

Table 10. User_roles table

Attribute	Data Type	Key	Constraints	Description
user_role_id	INT	PK	AUTO_INCREMENT, NOT NULL	Unique internal identifier for the role assignment record
user_id	INT	FK	NOT NULL, References users(user_id)	Foreign key referencing the assigned user account
role_id	INT	FK	NOT NULL, References roles(role_id)	Foreign key referencing the granted role

Description: Junction table managing the Many-to-Many (N:M) relationship between users and roles. It enables role-based access control (RBAC), allowing a single user account to hold multiple system roles simultaneously (e.g., a teacher who is also an administrator).

Table 11. Subjects table

Attribute	Data Type	Key	Constraints	Description
subject_id	INT	PK	AUTO_INCREMENT, NOT NULL	Unique internal identifier for the subject
subject_code	VARCHAR(20)		UNIQUE, NOT NULL	Short academic code identifying the subject (e.g., MATH, PHYS, CHEM, ENG)
subject_name	VARCHAR(150)		NOT NULL	Full official name of the subject (e.g., Mathematics, Physics)
description	TEXT		NULL	Detailed description of the subject curriculum and learning objectives

Description: Academic catalog storing all subjects offered by the learning center. It serves as the foundation for creating course sections and assigning specialized teaching staff.

Table 12. Classes table

Attribute	Data Type	Key	Constraints	Description
class_id	INT	PK	AUTO_INCREMENT, NOT NULL	Unique identifier for the class section
class_name	VARCHAR(100)		NOT NULL	Name of the class (e.g., Class 7 - Mathematics)
subject_id	INT	FK	NOT NULL, References subjects(subject_id)	Identifier of the associated subject
room	VARCHAR(50)		NULL	Assigned physical or virtual classroom

tuition_fee	DECIMAL(12,2)		NOT NULL, Default: 0.00	Monthly tuition rate in VND
start_date	DATE		NULL	Class start date
end_date	DATE		NULL	Class end date
status	ENUM		'active', 'inactive', Default: 'active'	Operational status of the class

Description**:** Stores concrete class offerings associated with academic subjects, classroom information, tuition fees, operating periods, and class status.

Table 13. Teacher_classes table

Attribute	Data Type	Key	Constraints	Description
teacher_class_id	INT	PK	AUTO_INCREMENT, NOT NULL	Unique identifier for the teacher-class assignment
teacher_id	INT	FK	NOT NULL, References users(user_id)	Identifier of the assigned teacher
class_id	INT	FK	NOT NULL, References classes(class_id)	Identifier of the assigned class
assigned_at	DATETIME		NOT NULL, Default: CURRENT_TIMESTAMP	Date and time when the teacher was assigned to the class
status	ENUM		'active', 'inactive' Default: 'active'	Current status of the teacher-class assignment

Description: Manages teacher assignments to classes and supports multiple teachers across different classes.

Table 14. Ta_classes table

Attribute	Data Type	Key	Constraints	Description
ta_class_id	INT	PK	AUTO_INCREMENT, NOT NULL	Unique identifier for the teaching assistant-class assignment
ta_id	INT	FK	NOT NULL, References users(user_id)	Identifier of the assigned teaching assistant
class_id	INT	FK	NOT NULL, References classes(class_id)	Identifier of the assigned class
assigned_at	DATETIME		NOT NULL, Default: CURRENT_TIMESTAMP	Date and time when the teaching assistant was assigned to the class
status	ENUM		'active', 'inactive' Default: 'active'	Current status of the teaching assistant-class assignment

Description: Manages teaching assistant assignments to classes and supports multiple teaching assistants across different classes.

Table 15. Student_classes table

Attribute	Data Type	Key	Constraints	Description
sc_id	INT	PK	AUTO_INCREMENT, NOT NULL	Unique identifier for the enrollment record
student_id	INT	FK	NOT NULL, References users(user_id)	Identifier of the enrolled student
class_id	INT	FK	NOT NULL, References classes(class_id)	Identifier of the enrolled class
enrolled_date	DATE		NULL	Date the student was officially enrolled
status	ENUM		'active', 'dropped', 'completed', Default: 'active'	Current enrollment status of the student

Description**:** Junction entity recording student enrollments in class sections, managing the Many-to-Many (N:M) relationship between students and classes.

Table 16. Class_schedules table

Attribute	Data Type	Key	Constraints	Description
schedule_id	INT	PK	AUTO_INCREMENT, NOT NULL	Unique identifier for the schedule entry
class_id	INT	FK	NOT NULL, References classes(class_id)	Identifier of the scheduled class
day_of_week	ENUM		'monday', 'tuesday', 'wednesday', 'thursday', 'friday', 'saturday', 'sunday'	Day of the recurring weekly session
start_time	TIME		NOT NULL	Session start time
end_time	TIME		NOT NULL	Session end time
room	VARCHAR(50)		NULL	Assigned classroom for this specific time slot

Description: Stores recurring weekly timetable slots for class sections, specifying the day of the week, duration, and assigned room.

Table 17. Class_sessions table

Attribute	Data Type	Key	Constraints	Description
session_id	INT	PK	AUTO_INCREMENT, NOT NULL	Unique identifier for the class session

class_id	INT	FK	NOT NULL, References classes(class_id)	Identifier of the class
schedule_id	INT	FK	NULL, References class_schedules(schedule_id)	Related recurring schedule of the session
session_date	DATE		NOT NULL	Date on which the class session takes place
start_time	TIME		NOT NULL	Actual start time of the session
end_time	TIME		NOT NULL	Actual end time of the session
room	VARCHAR(50)		NULL	Room or location of the session
status	ENUM		'scheduled', 'completed', 'cancelled', Default: 'scheduled'	Current status of the class session

Description: Stores individual teaching sessions of each class, including the session date, time, location, and status.

Table 18. Content_units table

Attribute	Data Type	Key	Constraints	Description
content_unit_id	INT	PK	AUTO_INCREMENT, NOT NULL	Unique identifier for the learning content unit
class_id	INT	FK	NOT NULL, References classes(class_id)	Identifier of the class that owns the content unit
parent_unit_id	INT	FK	NULL, References content_units(content_unit_id)	Identifier of the parent content unit in the hierarchy
title	VARCHAR(255)		NOT NULL	Title of the learning content unit
unit_type	ENUM		'chapter', 'lesson', 'topic', NOT NULL	Type of the learning content unit
description	TEXT		NULL	Additional description of the learning content
order_index	INT		NOT NULL, Default: 0	Display order of the content unit

Description: Organizes learning content for each class in a hierarchical structure using parent-child relationships.

Table 19. Attendance table

Attribute	Data Type	Key	Constraints	Description
attendance_id	INT	PK	AUTO_INCREMENT, NOT NULL	Unique identifier for the attendance record
session_id	INT	FK	NOT NULL, References class_sessions(session_id)	Identifier of the class session in which attendance is recorded
student_id	INT	FK	NOT NULL, References users(user_id)	Identifier of the student being evaluated
status	ENUM		'present', 'late', 'absent', 'excuse'	Attendance status
note	TEXT		NULL	Justification or note regarding student attendance
recorded_by	INT	FK	NOT NULL, References users(user_id)	Identifier of the staff/teacher who recorded the entry

Description**:** Tracks daily student attendance per class session, capturing presence status, notes, and the authoring instructor.

Table 20. Assignments table

Attribute	Data Type	Key	Constraints	Description
assignment_id	INT	PK	AUTO_INCREMENT, NOT NULL	Unique identifier for the assignment
content_unit_id	INT	FK	NULL, References content_units(content_unit_id)	Identifier of the related learning content unit
class_id	INT	FK	NOT NULL, References classes(class_id)	Identifier of the class receiving the assignment
title	VARCHAR(255)		NOT NULL	Title of the assignment
description	TEXT		NULL	Detailed instructions and problem descriptions
file_url	VARCHAR(500)		NULL	URL pointing to the attached assignment document or resource
due_date	DATETIME		NOT NULL	Deadline for submission
max_score	DECIMAL(5,2)		NOT NULL, Default: 10.00	Maximum attainable score

created_by	INT	FK	NOT NULL, References users(user_id)	Identifier of the instructor who created the assignment

Description**:** Stores coursework, homework, and assessments issued by teachers for specific class sections.

Table 21. Submissions table

Attribute	Data Type	Key	Constraints	Description
submission_id	INT	PK	AUTO_INCREMENT, NOT NULL	Unique identifier for the submission record
assignment_id	INT	FK	NOT NULL, References assignments(assignment_id)	Identifier of the associated assignment
student_id	INT	FK	NOT NULL, References users(user_id)	Identifier of the submitting student
file_url	VARCHAR(500)		NULL	URL link pointing to the student's uploaded solution file
submitted_at	DATETIME		NULL	Timestamp of submission
score	DECIMAL(5,2)		NULL	Graded score awarded to the submission
feedback	TEXT		NULL	Academic comment given by the instructor regarding the student's work.
status	ENUM		'submitted', 'graded', 'late', Default: 'submitted'	The current evaluation status of the student's submission.
graded_by	INT	FK	NULL, References users(user_id)	Identifier of the teacher or teaching assistant who evaluated the submission.

Description**:** Records student submissions for issued assignments, storing uploaded solution files, submission timestamps, awarded scores, and instructor feedback.

Table 22. AI_evaluations table

Attribute	Data Type	Key	Constraints	Description
evaluation_id	INT	PK	AUTO_INCREMENT, NOT NULL	Unique identifier for the AI evaluation record

submission_id	INT	FK	NOT NULL, References submissions(submission_id)	Identifier of the submission being evaluated
requested_by	INT	FK	NOT NULL, References users(user_id)	Identifier of the teacher who initiated the AI grading
ai_score	DECIMAL(5,2)		NULL	Score suggested by the AI model
ai_feedback	TEXT		NULL	Qualitative feedback and suggestions generated by AI
raw_response	TEXT		NULL	Full response payload returned from the AI API
status	VARCHAR(20)		NOT NULL	Execution status (PENDING, SUCCESS, FAILED)
created_at	TIMESTAMP		NOT NULL, DEFAULT CURRENT_TIMESTAMP	Timestamp when the AI evaluation was requested

Description**:** Stores automated grading results and feedback from the AI API for student submissions, including suggested scores, comments, and request status for auditing.

Table 23. Progress_notes table

Attribute	Data Type	Key	Constraints	Description
note_id	INT	PK	AUTO_INCREMENT, NOT NULL	Unique identifier for the evaluation entry
student_id	INT	FK	NOT NULL, References users(user_id)	Identifier of the evaluated student
class_id	INT	FK	NOT NULL, References classes(class_id)	Identifier of the associated class section
written_by	INT	FK	NOT NULL, References users(user_id)	Identifier of the authoring instructor or assistant
content	TEXT		NOT NULL	Qualitative comments and academic evaluation content
note_type	ENUM		'feedback', 'milestone', 'warning', 'praise', Default: 'feedback'	Classification category of the note

created_at	DATETIME		NOT NULL, Default: CURRENT_TIMESTAMP	Timestamp when the note was recorded

Description**:** Stores qualitative evaluations, developmental milestones, and academic feedback recorded by instructors for individual students within specific class sections.

Table 24. Tuition_invoices table

Attribute	Data Type	Key	Constraints	Description
invoice_id	INT	PK	AUTO_INCREMENT, NOT NULL	Unique identifier for the tuition invoice
student_id	INT	FK	NOT NULL, References users(user_id)	Identifier of the billed student
class_id	INT	FK	NOT NULL, References classes(class_id)	Identifier of the associated class section
period_month	TINYINT		NOT NULL (1 - 12)	Billing period month
period_year	YEAR		NOT NULL	Billing period year
amount_due	DECIMAL(12,2)		NOT NULL	Total payable tuition amount in VND
due_date	DATE		NOT NULL	Payment deadline date
status	ENUM		'unpaid', 'paid', 'overdue', 'pending_adjustment', Default: 'unpaid'	Payment status of the invoice

Description**:** Stores monthly tuition fee claims generated for individual students per enrolled class section, tracking payable amounts, deadlines, and settlement status.

Table 25. Payment table

Attribute	Data Type	Key	Constraints	Description
payment_id	INT	PK	AUTO_INCREMENT, NOT NULL	Unique identifier for the payment transaction
invoice_id	INT	FK	NOT NULL, References tuition_invoices(invoice_id)	Foreign key identifying the tuition invoice associated with this payment.
amount	DECIMAL(12,2)		NOT NULL	Actual monetary amount settled
payment_method	ENUM		'cash', 'bank_transfer', 'vnpay', 'momo'	Channel utilized for payment

transaction_code	VARCHAR(100)		NULL	Transaction reference code from banking gateway or digital wallet
paid_at	DATETIME		NOT NULL, Default: CURRENT_TIMESTAMP	Timestamp of payment execution
paid_by_user_id	INT	FK	NULL, References users(user_id)	Identifier of the student who completed payment via online portal
recorded_by	INT	FK	NULL, References users(user_id)	Identifier of the staff member who collected payment at front desk
Note	VARCHAR(255)		NULL	Additional notes or remarks about the payment transaction.

Description**:** Stores payment transactions associated with tuition invoices, allowing an invoice to contain multiple payment records and preserving payment details for financial tracking.

Table 26. Approval_requests table

Attribute	Data Type	Key	Constraints	Description
request_id	INT	PK	AUTO_INCREMENT, NOT NULL	Unique identifier for the approval request
request_code	VARCHAR(20)		UNIQUE, NOT NULL	Human-readable tracking code (e.g., AR0001)
request_type	ENUM		'tuition_adjustment', 'refund', 'credit', 'due_date_extension', 'leave_request'	Nature of the exception
student_id	INT	FK	NULL, References users(user_id)	Identifier of the associated student
invoice_id	INT	FK	NULL, References tuition_invoices(invoice_id)	Identifier of the targeted invoice (if applicable)
class_id	INT	FK	NULL, References classes(class_id)	Identifier of the targeted class section (if applicable)
requested_by	INT	FK	NOT NULL, References users(user_id)	Identifier of the staff/teacher initiating the request
reviewed_by	INT	FK	NULL, References users(user_id)	Identifier of the manager who reviewed the request

priority	ENUM		'high', 'medium', 'low', Default: 'medium'	Urgency level of the request
status	ENUM		'pending', 'approved', 'rejected', 'cancelled', Default: 'pending'	Processing status of the request
reason	TEXT		NULL	Detailed explanation and justification for the request

Description**:** Manages administrative exception requests (tuition discounts, refunds, due date extensions, or leave applications) submitted by staff or teachers for Center Manager review.

Table 27. Notifications table

Attribute	Data Type	Key	Constraints	Description
notification_id	INT	PK	AUTO_INCREMENT, NOT NULL	Unique identifier for the notification
user_id	INT	FK	NULL, References users(user_id)	Recipient user identifier (NULL indicates a center-wide broadcast)
Class_id	INT	FK	NULL, References classes(class_id)	Target class identifier (NULL indicates the notification is not restricted to a specific class)
title	VARCHAR(255)		NOT NULL	Title of the notification
content	TEXT		NULL	Full message content
type	ENUM		'class', 'system', 'tuition', 'schedule', 'homework', 'feedback', Default: 'system'	Classification category of the notification
is_read	BOOLEAN		NOT NULL, Default: FALSE	Read status indicator
created_at	DATETIME		NOT NULL, Default: CURRENT_TIMESTAMP	Timestamp when the notification was created

Description**:** Stores system announcements, automated reminders, and operational alerts, supporting both broadcast (center-wide) and user-targeted notifications.

Table 28. Center_settings table

Attribute	Data Type	Key	Constraints	Description
setting_id	INT	PK	AUTO_INCREMENT, NOT NULL	Unique identifier for the setting entry

setting_key	VARCHAR(100)		UNIQUE, NOT NULL	Unique configuration key (e.g., center_name, academic_year, tuition_due_day)
setting_value	TEXT		NULL	Stored configuration value
updated_by	INT	FK	NULL, References users(user_id)	Identifier of the manager who last modified the configuration

Description**:** Centralized storage for global configuration parameters, system operational values, and learning center information.

3.6. Feature and Page Analysis

This section outlines the essential pages required for the EduTrack system, categorized by user roles. To maintain a concise architecture, the analysis focuses on the core functionalities, primary actions, and key UI elements of each page.

Table 29. Center Manager & Admin Staff Pages

No.	Path	Page Name	Key Features & UI Elements
1	/cms/login	Login	Authentication form (username/password), "Remember me" checkbox, and account recovery links.
2	/cms/dashboard	Dashboard	Overview metrics, summary cards (active students, classes, tuition), and pending request alerts.
3	/cms/users	User Accounts	Manager only: Data table to manage staff/teachers, assign roles, change status, and reset passwords.
4	/cms/students	Student Management	Data table for students. Actions: Add new, view/edit profiles, and update enrollment statuses.
5	/cms/subjects	Subject Management	Manager only: Data table to create, edit, and manage the center's subjects.
6	/cms/classes	Class Management	Manager: Create and manage classes and their Class Subjects; assign Teachers/TAs to Class Subjects. Staff: Manage student enrollment and operational information at the Class Subject level.
7	/cms/tuition	Tuition Management	Data table of tuitions. Actions: Generate bills, record payments, filter by status (Paid/Overdue), and send reminders.
8	/cms/approvals	Approval Requests	Data table for exception requests (e.g., fee waivers, refunds). Staff: Submit and edit drafts. Manager: Review, approve, or reject requests.

9	/cms/teaching-monitoring	Teaching Monitoring	Staff: Track teacher schedules, monitor class participation rates, and observe student progress.
10	/cms/notifications	Notifications	Compose messages (draft/schedule/send), view history, and filter by recipient groups.
11	/cms/reports	Reports	Generate, view, and export (Excel/PDF) statistical reports. Manager: Includes full revenue and financial data. Both: Students, classes, and basic tuition reports.
12	/cms/settings	Center Settings	Manager only: Configure center information, academic terms, and tuition/notification policies.

The system enforces a clear hierarchy between these two roles to ensure data security and operational control:

- Strategic Control (Center Manager)**:** Acts as the ultimate decision-maker. Only the Manager has the authority to configure system settings, manage employee accounts, access full revenue reports, and approve financial exceptions (e.g., fee waivers).
- Operational Execution (Admin Staff)**:** Focuses on the day-to-day operations of the center. Staff members handle student enrollments, record standard tuition payments, and monitor daily teaching activities, but they must escalate any rule exceptions to the Manager for final approval.

Table 30. Teacher & Teaching Assistant Pages

No.	Path	Page Name	Key Features & UI Elements
1	/cms/login	Login	Authentication form (username/password), "Remember me" checkbox, and account recovery links.
2	/cms/dashboard	Dashboard	Quick stats (today's teaching/assisting shifts, active assignments), new notifications, and a list of today's shifts.
3	/cms/classes	My Classes	Class list and detailed views (student list, attendance tracking). Teacher: Create, manage, and grade assignments. TA: Grade or check submitted assignments only when permission is granted by the Teacher.
4	/cms/schedule	Schedule	Weekly/monthly calendar view displaying shifts. Clicking a shift shows details (room, class size, and the assigned main Teacher/TA).
5	/cms/learning-progress	Learning Progress	View student attendance rates, assignment submission status, and average scores.

			Teacher only: Send academic feedback directly to students.
6	/cms/request	Requests	Create, edit, delete, and submit draft requests; track submitted requests and withdraw pending requests when applicable.
7	/cms/profile	Profile	View and edit personal information (avatar, phone number, email, address).
8	/cms/notifications	Notifications	Inbox to view system announcements, with filtering and sorting features based on date and status.

Although Teachers and Teaching Assistants (TAs) share the same user interface, the system enforces strict permission levels based on their roles:

- Assignment Management**:** Teachers have full authority to create and manage assignments. TAs are restricted to only grading and checking submitted work.
- Academic Feedback**:** Only Teachers are authorized to send official learning feedback and evaluations directly to students. TAs can view the progress but cannot send feedback.
- Shift Details**:** The shared Schedule and Dashboard pages dynamically adapt to show TAs which main Teacher they are assisting for a specific shift.

Table 31. Student Pages

No.	Path	Page Name	Key Features & UI Elements
1	/cms/login	Login	Authentication form (username/password), "Remember me" checkbox, center contact info for new accounts, and password recovery links.
2	/cms/dashboard	Dashboard	Quick stats (total enrolled classes, impending assignments) and a summary list of regular weekly classes.
3	/cms/myClasses	My Classes	List of currently enrolled classes, displaying the assigned teacher and weekly schedule.
4	/cms/myClasses/classDetail	Class Detail	Detailed class information, downloadable learning materials, and a list of assignments with due dates.
5	/cms/myClasses/classDetail/assignmentDetail	Assignment Detail	View assignment instructions/resources, and submit, edit, or remove homework (including interactive quizzes).
6	/cms/schedule	Schedule	Weekly calendar view displaying class timings, locations, assigned teachers, and session statuses.

7	/cms/progress	Progress	Attendance statistics and a detailed table of assignment feedback/grades for both current and past classes.
8	/cms/tuition	Tuition	View current billing details, payment history, and execute full online tuition payments.
9	/cms/notifications	Notifications	Inbox for system messages, with filtering (by type/status) and sorting capabilities.
10	/cms/profile	Profile	View and edit personal information (name, DOB, address, phone, email) and avatar.

To optimize the system's scope and streamline the user experience for the initial development phase, a standalone Parent portal has been excluded. Instead, the system adopts a **Shared Access Model** where Parents utilize the Student's account credentials to log in.

This unified interface perfectly serves both user groups without redundancy: Students use it daily to submit assignments and check schedules, while Parents can periodically log into the same account to monitor attendance statistics (/cms/progress), read teacher feedback, and execute online payments (/cms/tuition).

Table 32. Landing Page

No.	Path	Page Name	Key Features & UI Elements
1	/ (Root)	Landing Page	Center introduction, course catalog, teacher profiles, student achievements, and contact information.

Before users log into the system, they interact with the public landing page, which serves as the digital storefront of the center

CHAPTER 4. RESULTS & DISCUSSION

This chapter presents the static user interface designs (UI Mock-ups) for the core pages of the EduTrack system. These prototypes illustrate the expected layout, data presentation, and user flow before entering the development phase.

4.1. Center Manager and Admin Staff UIs

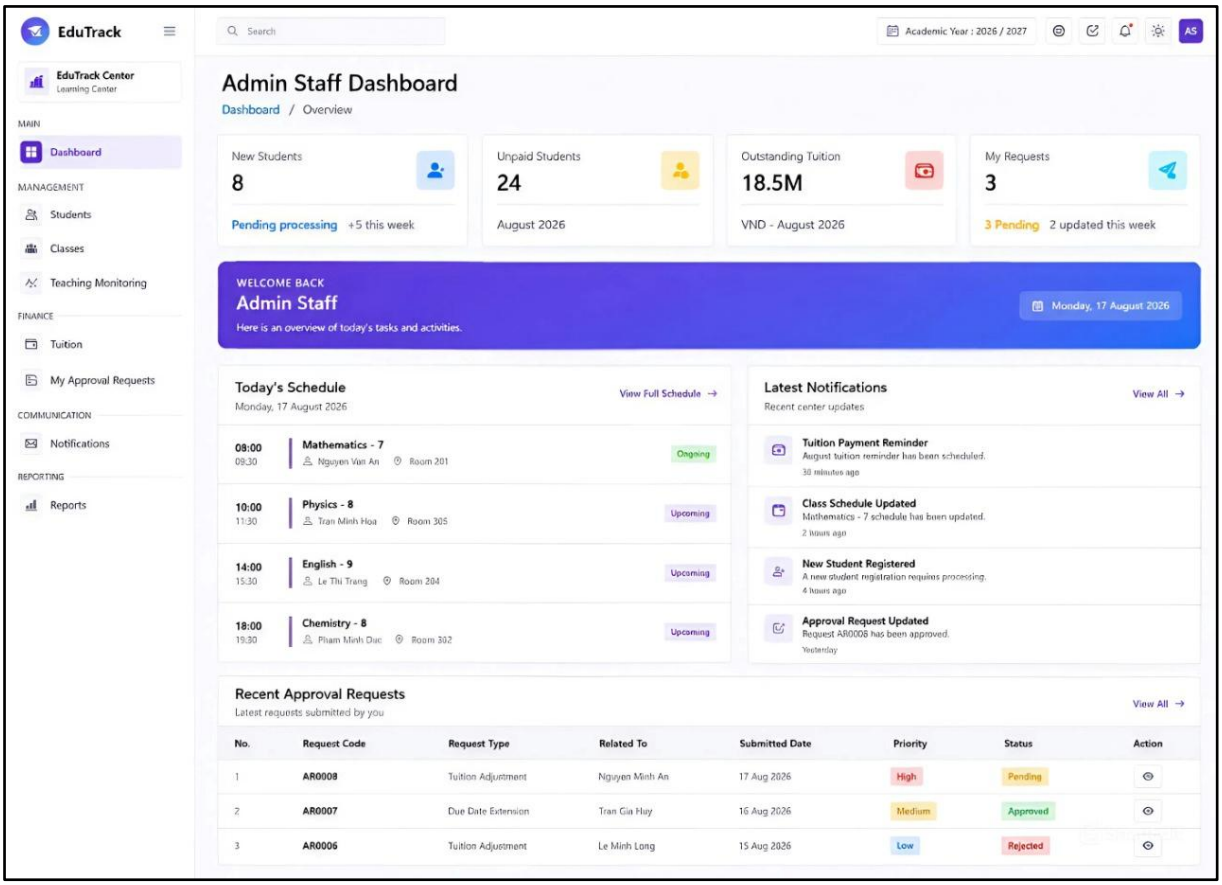

Figure 9. UI Prototype of Admin Staff Dashboard

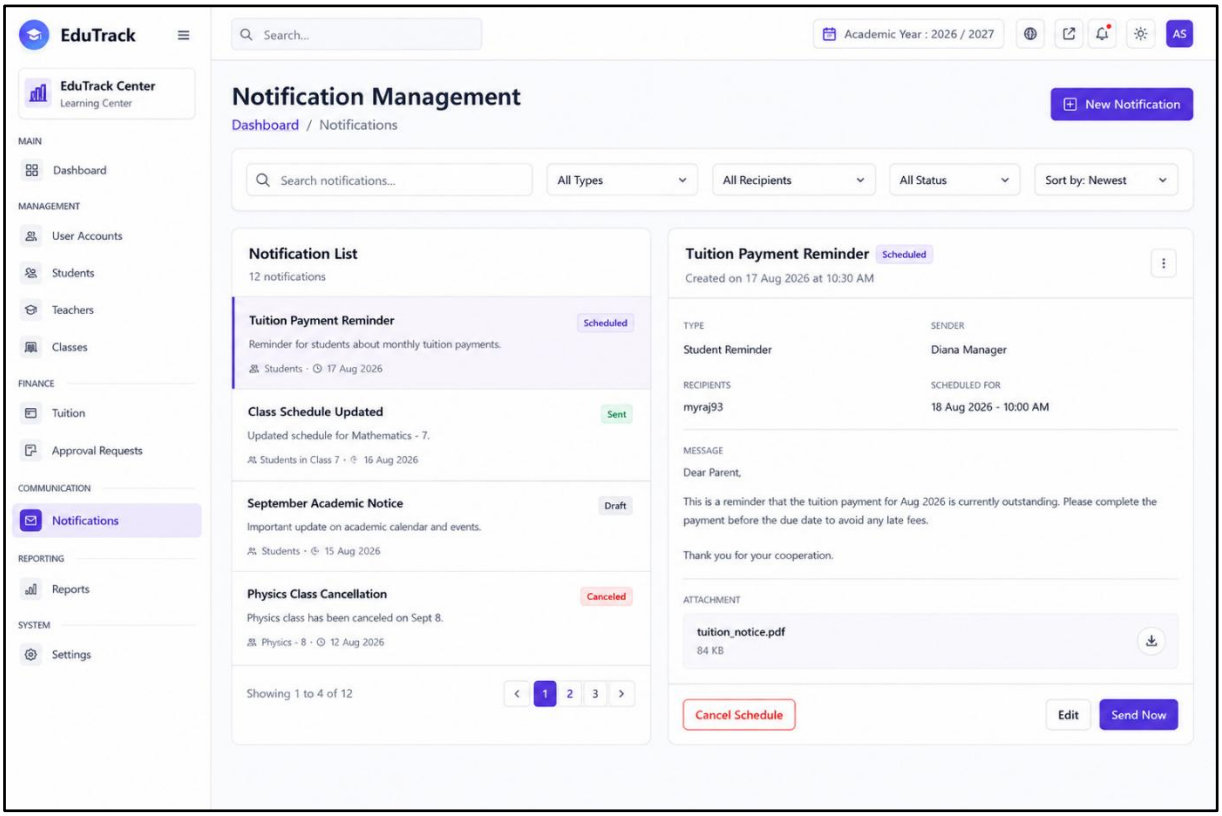

Figure 10. UI Prototype of Center Manager Notification Management

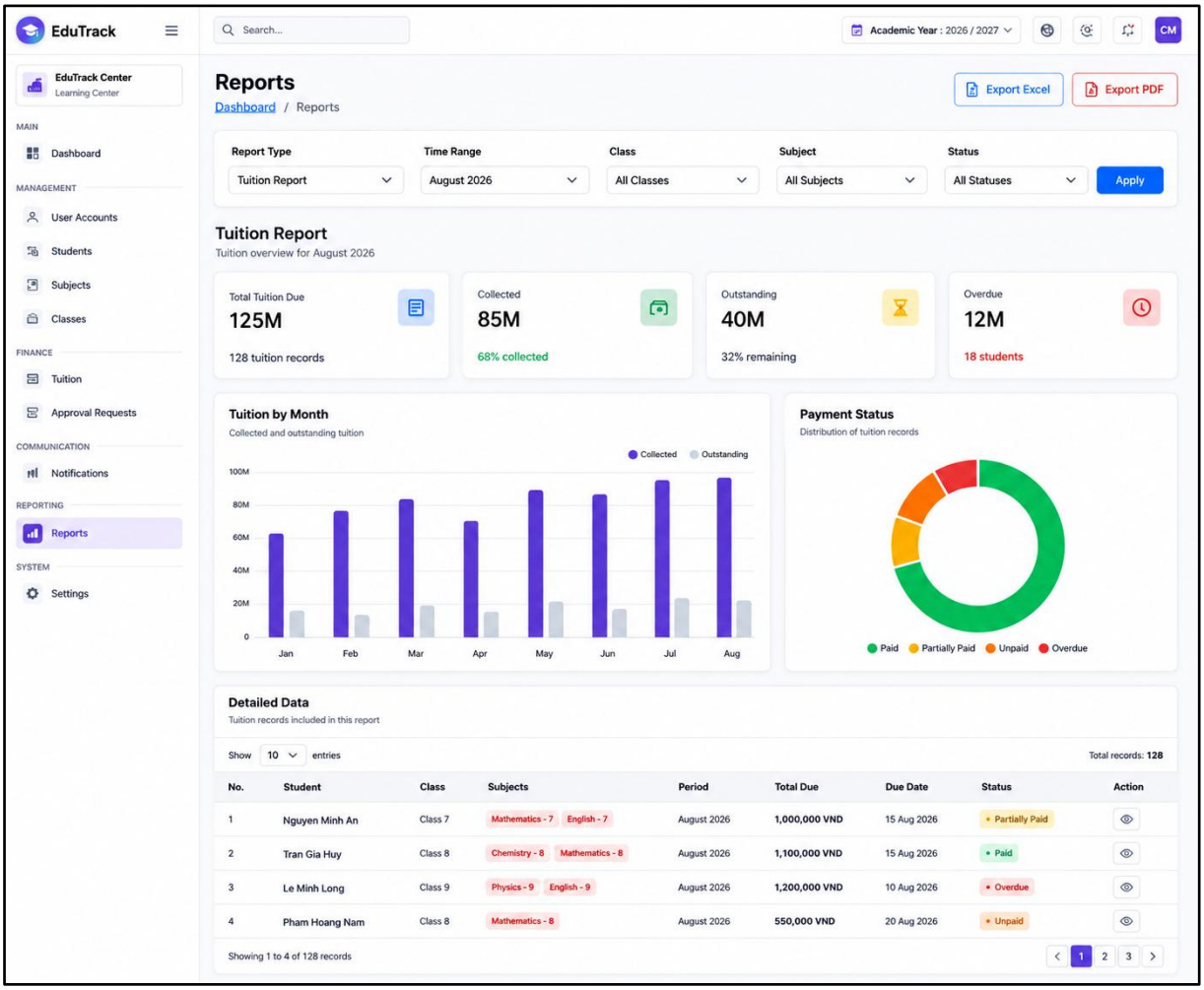

Figure 11. UI Prototype of Tuition Report

4.2. Teacher and TA UIs

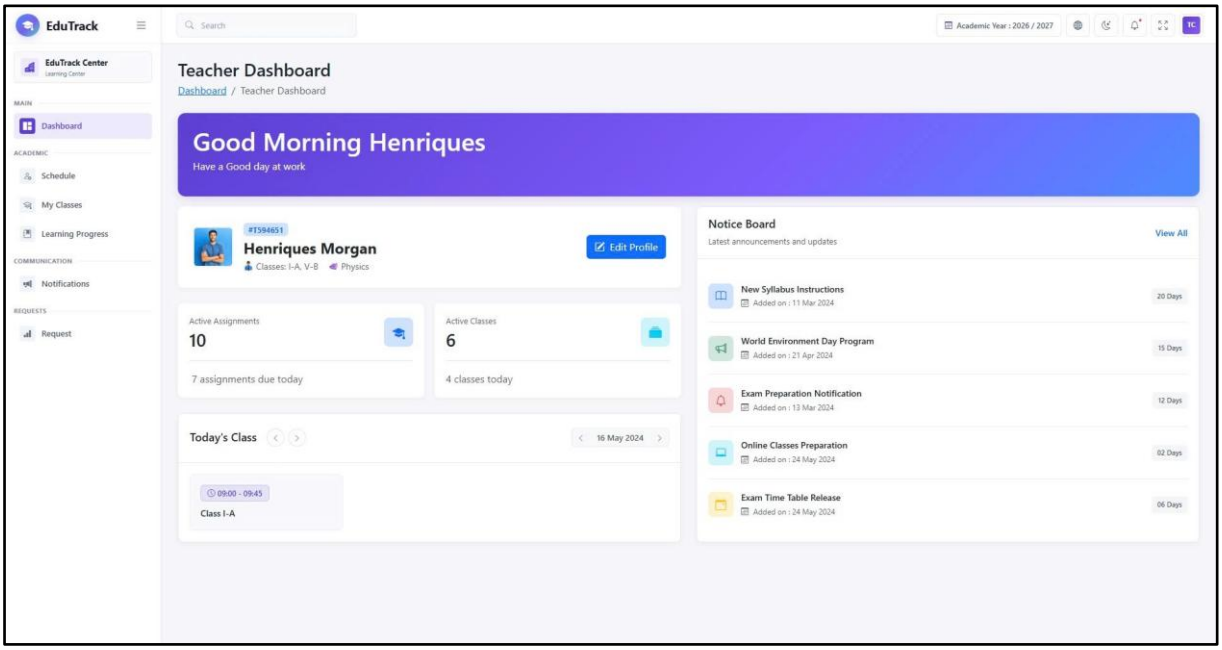

Figure 12. Teacher Dashboard Prototype

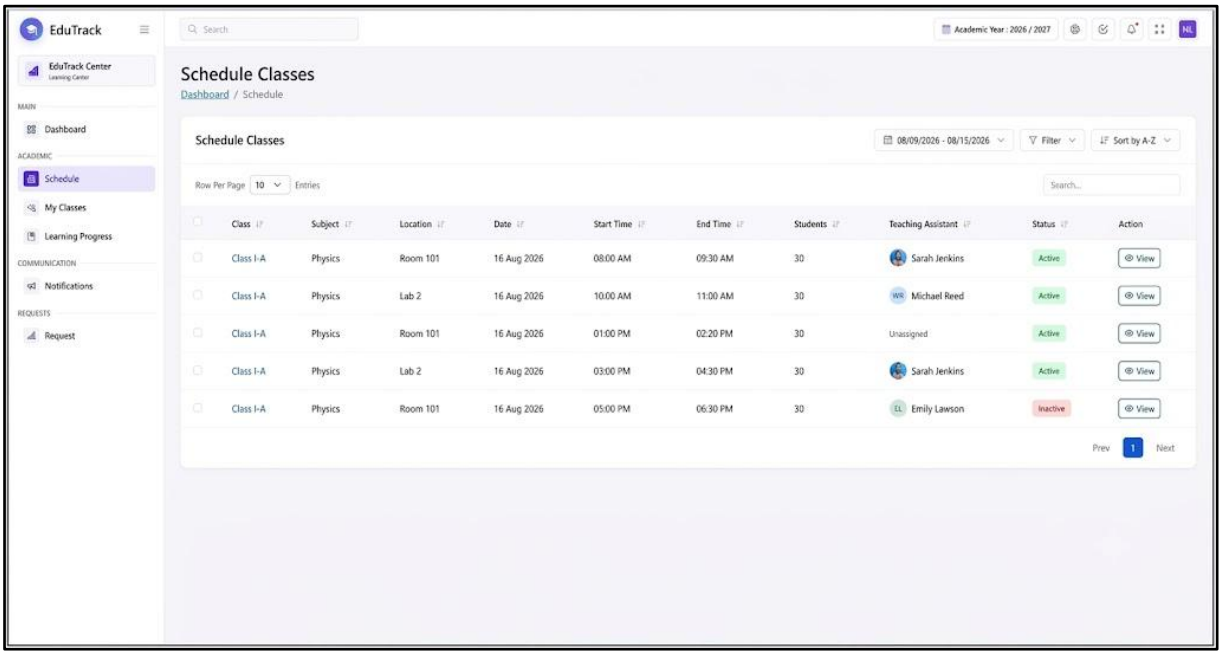

Figure 13. Class Schedule Interface Prototype

4.3. Student UIs

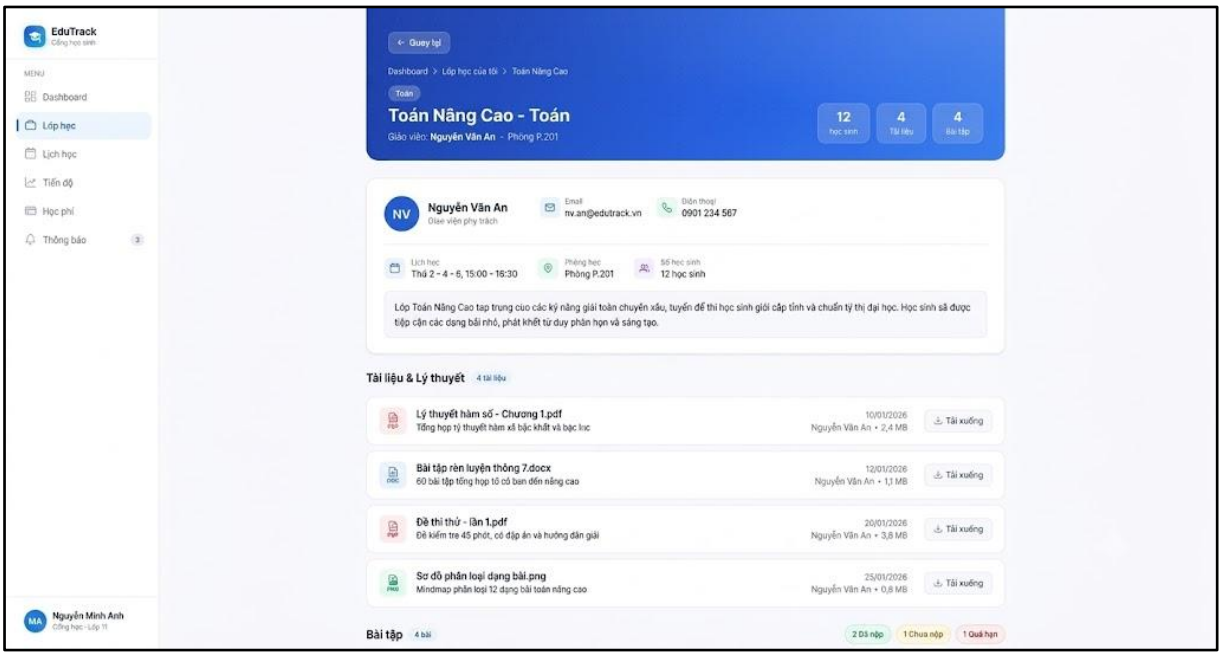

Figure 14. Class Detail interface for Students

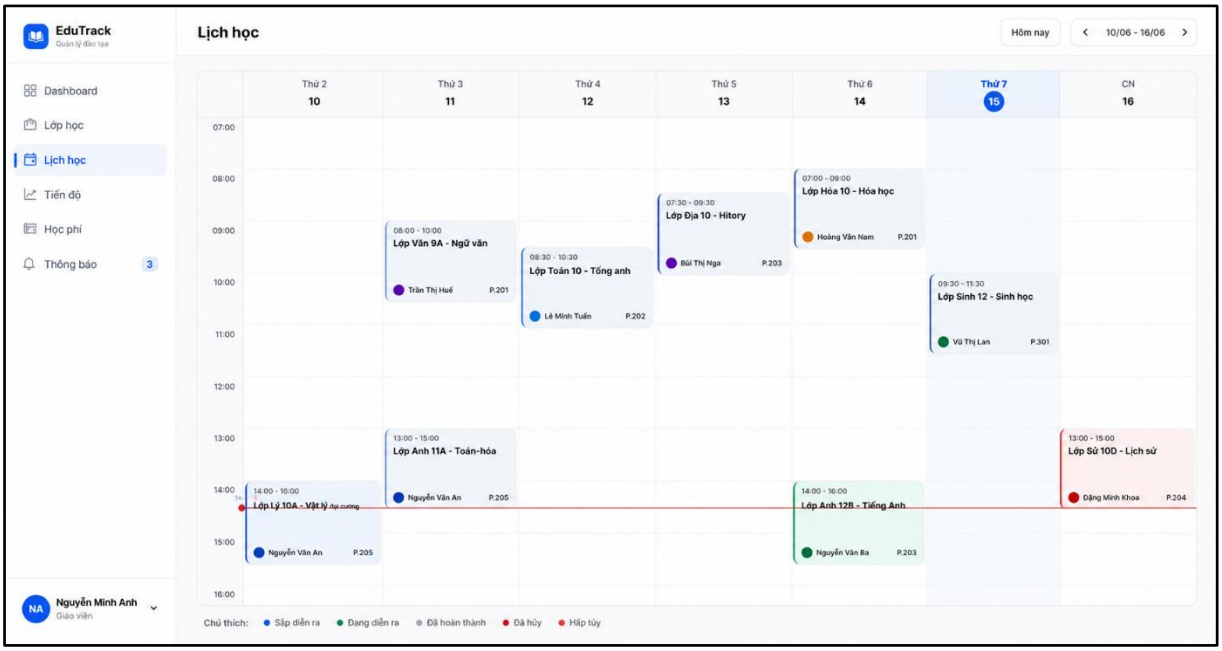

Figure 15. Interactive weekly schedule interface.

CHAPTER 5. CONCLUSION & FUTURE WORKS

5.1. Conclusion

Throughout the development of Capstone Project I, our team has achieved significant progress and learned valuable lessons across multiple areas:

- Knowledge and Tools: We gained a clear understanding of foundational software engineering processes, from gathering requirements to designing system architectures. We also learned how to use essential practical tools such as Git for version control, and MySQL Workbench for database design.
- Product Value: We successfully analyzed the theoretical foundation for EduTrack. Although the project is currently in the analysis phase, the proposed system clearly outlines how it will help educational centers manage students, classes, and tuitions efficiently, replacing slow and error-prone manual paperwork.
- Technical Skills: We significantly improved our problem-solving and analytical skills. We learned how to translate real-world user needs into technical diagrams (such as Use Case and Activity diagrams) and how to build a logical, well-structured relational database (ERD).
- Soft Skills: Working closely together as a three-member team, we developed our communication and collaboration skills. We learned how to divide tasks effectively, resolve technical disagreements, and manage our time to meet project deadlines.

5.2. Future Works

Since Capstone Project I primarily focuses on system analysis, database design, and UI prototyping, the next phase in Capstone Project II will be dedicated to the technical implementation of the EduTrack system. Specifically, the future development plan will encompass the full-stack realization—from Front-end interfaces to Back-end logic—of all the functional features and business workflows analyzed in this report.

REFERENCES

[1] UNESCO, "Regulating private tutoring for the public good," *UNESCO*, Dec. 09, 2021. [Online]. Available: https://www.unesco.org/en/articles/regulating-private-tutoring-public-good. [Accessed: Aug. 2026].

[2] Báo Chính Phủ, "Chuyển đổi số trường học với giải pháp hồ sơ số giáo dục và học bạ điện tử," *Báo điện tử Chính phủ*, Sep. 13, 2022. [Online]. Available: https://baochinhphu.vn/chuyen-doi-so-truong-hoc-voi-giai-phap-ho-so-so-giao-duc-va-hoc-ba-dien-tu-102230913153327932.htm. [Accessed: Aug. 2026].

[3] TutorBird, "TutorBird: Tutor & Music Teacher Management Software," *TutorBird*. [Online]. Available: https://www.tutorbird.com/. [Accessed: Aug. 2026].

[4] Easy Edu, "Phần mềm quản lý trung tâm giáo dục, đào tạo Easy Edu," *Easy Edu*. [Online]. Available: https://easyedu.vn/. [Accessed: Aug. 18, 2026].

[5] EduSpace, "Phần mềm quản lý trung tâm đào tạo, ngoại ngữ toàn diện EduSpace," *EduSpace*. [Online]. Available: https://eduspace.vn/. [Accessed: Aug. 2026].

[6] Moodle HQ, "Moodle: Open-source learning platform," *Moodle*. [Online]. Available: https://moodle.org/. [Accessed: Aug. 2026].

[7] Meta Open Source, "React – The library for web and native user interfaces," *React.dev*, 2024. [Online]. Available: https://react.dev/. [Accessed: Aug. 2026].

[8] Microsoft, "TypeScript: JavaScript With Syntax For Types," *TypeScriptlang.org*, 2024. [Online]. Available: https://www.typescriptlang.org/. [Accessed: Aug. 2026].

[9] Microsoft, "ASP.NET Core Documentation," *Microsoft Learn*, 2024. [Online]. Available: https://learn.microsoft.com/en-us/aspnet/core/. [Accessed: Aug. 2026].

[10] Oracle Corporation, "MySQL Documentation," *MySQL.com*, 2024. [Online]. Available: https://dev.mysql.com/doc/. [Accessed: Aug. 2026].

[11] Auth0, "JSON Web Tokens," *JWT.io*. [Online]. Available: https://jwt.io/. [Accessed: Aug. 2026].